JAVA 2

Module ABS

Classes abstraites

Au programme

1. L'héritage en Java
2. Le même code, trois fois
3. La classe abstraite
4. Ce qui monte, ce qui descend
5. Le polymorphisme
6. La mère appelle la fille
7. Deux étages
8. Décider soi-même

1. L'héritage en Java



Une classe qui reprend une autre

Le mot-clé `extends` place une classe sous une autre

La sous-classe reçoit les champs et les méthodes de la classe mère, sans les réécrire

Elle ajoute ce qui lui est propre

```
public class CompteEpargne extends CompteBancaire {

    private final double taux;

    public void crediterInterets() {
        deposer(getSolde() * taux);
    }
}
```

Appeler le constructeur de la classe mère

Le mot-clé `super` appelle le constructeur de la classe mère

Cet appel est toujours la première instruction du constructeur

La classe mère initialise sa part, la classe fille initialise la sienne

```
public class CompteEpargne extends CompteBancaire {

    public CompteEpargne(long numero, double taux) {
        super(numero);
        this.taux = taux;
    }
    ...
}
```

Redéfinir une méthode

Une sous-classe peut redéfinir une méthode héritée en la réécrivant

L'annotation `@Override` demande au compilateur de vérifier qu'il s'agit bien d'une redéfinition

Elle est facultative, et il faut toujours l'écrire

```
public class CompteEpargne extends CompteBancaire {
    ...
    @Override
    public void deposer(double montant) {
        super.deposer(montant);
        journaliser(montant);
    }
}
```

Ce que la fille voit de la mère

Un champ `private` de la mère reste invisible dans la fille

Le modificateur `protected` ouvre le membre aux sous-classes, et aussi au package.

Un accesseur `public` reste une solution, quand l'information regarde tout le monde

```
public class CompteBancaire {

    private double solde;          // invisible dans CompteEpargne
    protected long numero;         // visible dans CompteEpargne
}
```

De C# à Java

En C#	En Java
`class Fille : Mere`	`class Fille extends Mere`
`: base(...)`	`super(...)`, première instruction du constructeur
`override`	`@Override`, facultative (mais bonne pratique)
`is`	`instanceof`
`abstract`	`abstract`

A une personne, ou est une personne ?

Au module TAB, un vendeur **avait une** `Person` : un champ. L'héritage permet l'autre modèle : `Vendor extends Person`

Est une : le vendeur reçoit tout ce que sait la personne. Simple, tant que ce rôle ne change pas

A une : le vendeur est un rôle. La même personne peut aussi être cliente, ou cesser de vendre

```
// a une personne : un rôle
class Vendor {
    private Person personne;
    private Sale[] ventes;
}
```

```
// est une personne : l'héritage
class Vendor extends Person {
    private Sale[] ventes;
}
```



2. Le même code, trois fois

Un exemple : prévenir un utilisateur

Une application doit prévenir ses utilisateurs, et elle a plusieurs moyens de le faire

Un mail, un SMS, une notification sur le téléphone : trois canaux, un même geste : envoyer un texte

Première idée, la plus naturelle : une classe par canal, chacune écrite pour elle-même

Voyons ce que cette première idée coûte, avant de chercher mieux

```
NotificationMail
envoyer(texte)
```

```
NotificationSms
envoyer(texte)
```

```
NotificationPush
envoyer(texte)
```

Trois classes, un seul comportement

Trois canaux, trois classes indépendantes

La méthode d'envoi y est écrite trois fois

Une règle nouvelle devra être portée dans chacune

```java
public class NotificationMail {
    public void envoyer(String texte) {
        System.out.println("mail : " + texte);
    }
}
```

```java
public class NotificationSms {
    public void envoyer(String texte) {
        System.out.println("sms : " + texte);
    }
}
```

```java
public class NotificationPush {
    public void envoyer(String texte) {
        System.out.println("push : " + texte);
    }
}
```

Ce qu'on ne peut pas faire

Ranger les trois notifications dans un tableau demande un type commun

Ces trois classes n'en ont aucun, aux yeux du compilateur

Sans type commun, pas de boucle : chaque canal se traite à la main

```
NotificationMail[] canaux = { mail, sms, push };

// refus : sms et push ne sont pas des NotificationMail
```

Démonstration



Démonstration

Laisser l'outil supprimer la duplication

Déroulé : Demo_ABS

Le geste, pas le code

Le refactoring `Extract Superclass` crée la classe mère et y déplace ce qu'on désigne

Le refactoring `Pull Members Up` fait remonter une méthode d'une classe existante

L'outil déplace, corrige les déclarations et ne se trompe pas de nom

3. La classe abstraite



Une classe qu'on ne peut pas instancier

Le mot-clé `abstract` interdit la création d'objets de cette classe

On peut toujours déclarer une variable de ce type

Seules les sous-classes concrètes se construisent

```
public abstract class Notification { ... }

Notification envoi;                    // d'accord
envoi = new Notification("Ada");       // refus a la compilation
envoi = new NotificationMail("Ada"); // d'accord
```

À quoi sert un type qu'on ne construit pas

Il sert de type de variable, de paramètre et de valeur de retour

Il sert surtout de type d'élément dans un tableau (et plus tard dans une collection, voir COL)

Il désigne ce que toutes les sous-classes savent faire, et rien de plus

```
Notification[] canaux = {
    new NotificationMail("Ada"),
    new NotificationSms("0470..."),
    new NotificationPush("ada42")
};
```

Un constructeur, quand même

Une classe abstraite a un constructeur, même si personne ne l'appelle directement

Le constructeur de la sous-classe l'appelle par `super`

Le déclarer `protected` dit qu'il n'est là que pour les sous-classes.

```
public abstract class Notification {

    protected final String destinataire;

    protected Notification(String destinataire) {
        this.destinataire = destinataire;
    }
}
```

4. Ce qui monte, ce qui descend



La méthode abstraite

Une méthode abstraite se déclare par son entête, suivi d'un point-virgule

Elle n'a pas de corps : la classe mère ne sait pas comment faire

Une classe qui contient une méthode abstraite est elle-même abstraite

```
public abstract class Notification {

    protected abstract String canal();
}
```

La règle de partage

Une méthode dont le code serait identique partout appartient à la classe mère

Une méthode dont le code diffère à chaque fois est abstraite

Le reste du module n'est que l'application de ce principe

```java
public abstract class Forme {

    // ce qui VARIE : chaque sous-classe le definit
    public abstract String nom();

    // ce qui est COMMUN : ecrit une seule fois
    public void affiche() {
        System.out.println("Je suis un " + nom());
    }
}
```



L'obligation faite aux sous-classes

Une sous-classe concrète a l'obligation de définir toutes les méthodes

Sinon elle est abstraite à son tour, et elle doit le dire dans son en-tête

Le compilateur ne laisse pas passer l'oubli

```
public class Cercle extends Forme {

    @Override
    public String nom() {
        return "cercle";
    }
}
```

`@Override`, aussi pour une méthode abstraite

Rappel : `@Override` demande au compilateur de vérifier qu'on redéfinit bien une méthode de la classe mère

Définir une méthode abstraite, c'est aussi la redéfinir : l'annotation s'écrit de la même façon

Avec elle, une faute de frappe est signalée sur la ligne fautive, et non sur la classe

```
public class Cercle extends Forme {

    @Override
    public String Nom() {          // refus : Nom() ne redéfinit rien
        return "cercle";
    }
}
```

Petit détour : le reste d'une division négative

En Java, le reste garde le signe du dividende : `-90 % 360` vaut `-90`

Un modulo seul ne ramène donc pas une valeur dans un intervalle positif

Ajouter le diviseur puis reprendre le modulo corrige le cas

```
-90 % 360                  // -90, et non 270

(-90 % 360 + 360) % 360 // 270
```

Exercices

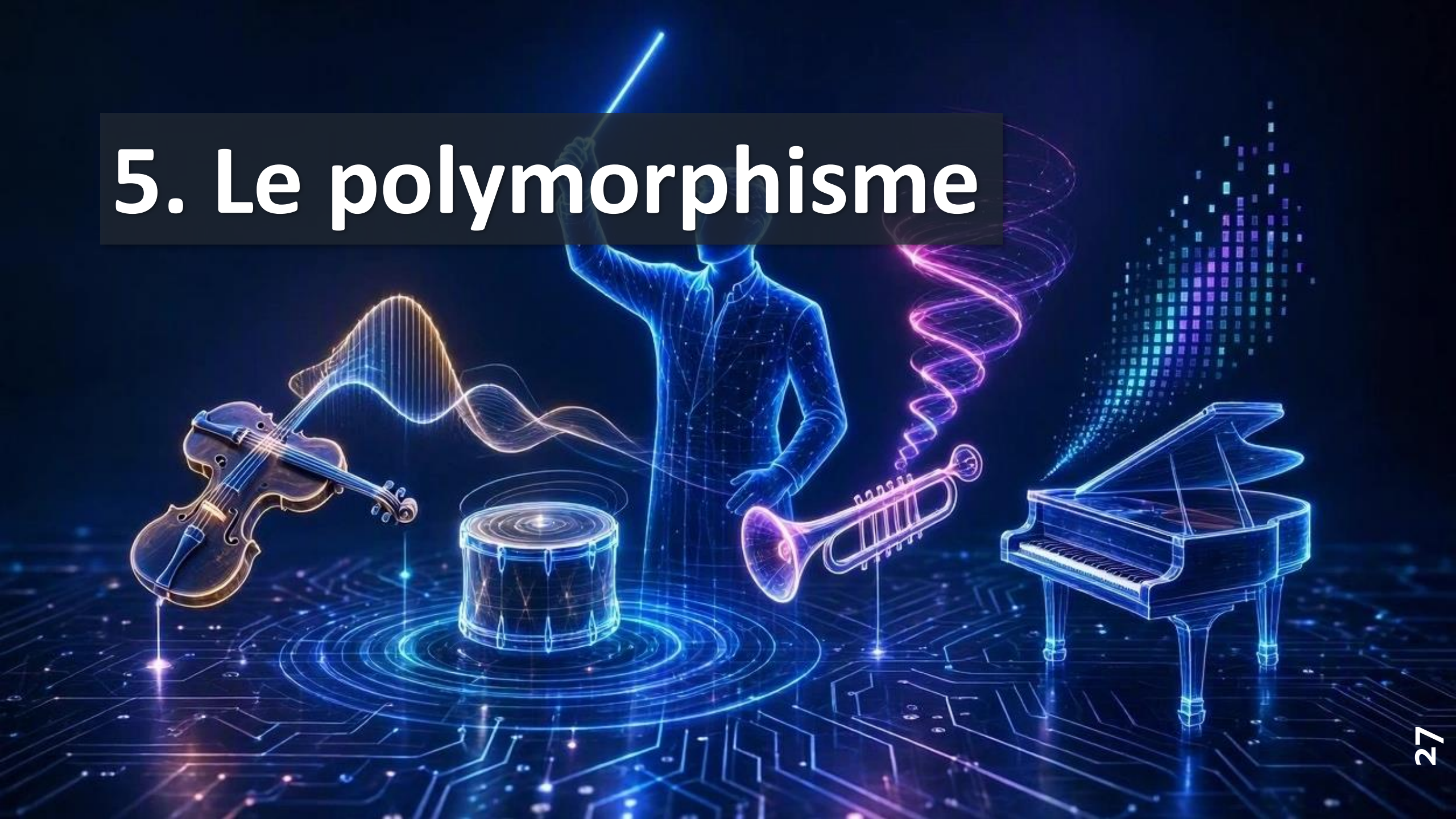


ABS.1 Des formes géométriques

fascicule ABS

Concevoir : CC-CLS.3

5. Le polymorphisme



Un tableau du type le plus général

Le tableau du code suivant est déclaré `Shape[]` et ne contient aucun `Shape`

Chaque case du tableau désigne un objet d'une sous-classe concrète

La boucle traite les trois formes sans savoir lesquelles elle traite

```
Shape[] formes = {
    new Circle(centre), new Square(centre), new Triangle(centre)
};

for (Shape forme : formes) {
    forme.affiche();
}
```

Qui répond à l'appel

La méthode exécutée est celle de la classe réelle de l'objet

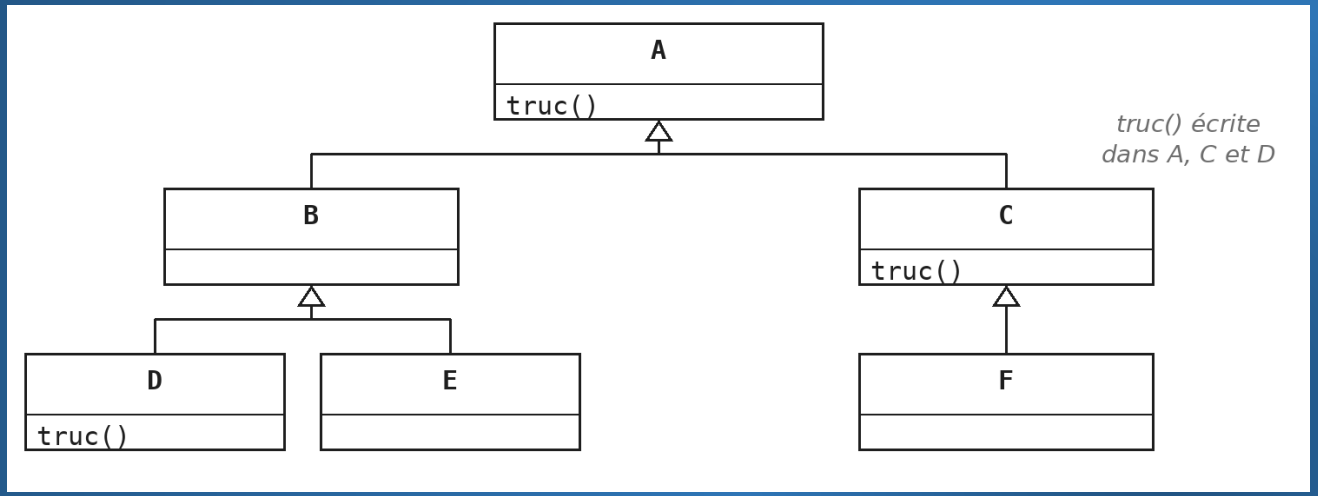
Le choix se fait à l'exécution, pas à la compilation

C'est le polymorphisme, annoncé au module POO

```
Shape forme = new Circle(centre);

forme.affiche();   // "Je suis un cercle et je m'affiche"
```

Quelle version s'exécute ?



On part de la classe réelle de l'objet

On remonte jusqu'à la première classe qui écrit la méthode

`super.truc()` : la version la plus proche au-dessus

Ni `instanceof`, ni test de type

Un test de type dans la boucle signale une hiérarchie mal découpée

Ce que vous testez, la machine virtuelle le sait déjà

Ajouter une sorte de `Shape` nécessite une classe, et aucune autre modification ; si vous utilisez un `instanceof` vous devrez faire d'autres modifications

```
for (Shape forme : formes) {
    if (forme instanceof Circle) {   // a ne pas ecrire
        ...
    }
}
```

Exercices



ABS.2 Des animaux qui font du bruit

fascicule ABS

Concevoir : CC-CLS.4

6. La mère appelle la fille

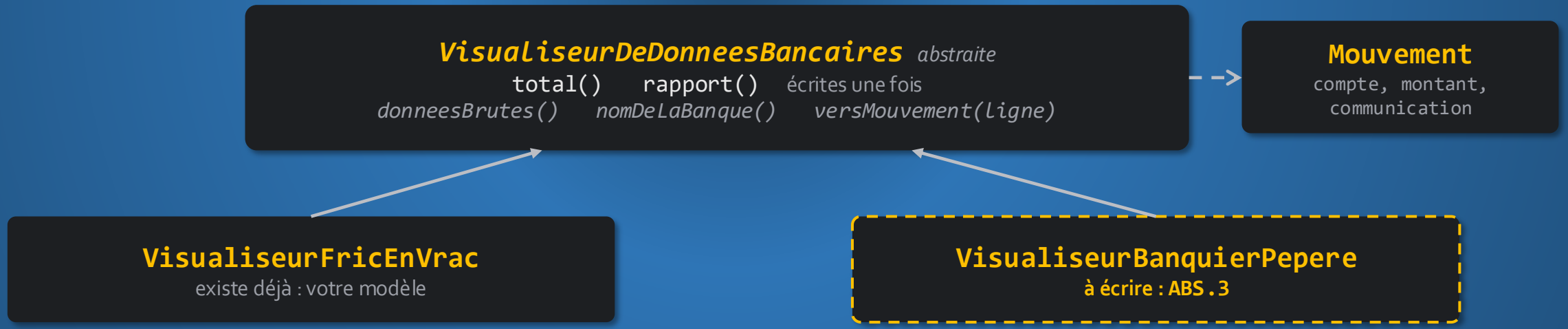


De quoi il s'agit

Une application affiche les mouvements bancaires d'une seule banque, Fric en Vrac

Il faut ajouter Banquier Pépère : mêmes informations, autre ordre de colonnes

Le total et le rapport restent les mêmes d'une banque à l'autre



Le squelette, puis le détail

La classe mère écrit le traitement entier

Elle déclare abstraits les seuls points qui varient

Une méthode concrète appelle une méthode abstraite

```
public abstract class VisualiseurDeDonneesBancaires {

    // a fournir par chaque banque
    protected abstract String[][] donneesBrutes();
    protected abstract String nomDeLaBanque();
    protected abstract Mouvement versMouvement(String[] ligne);

    // deja ecrit, une fois pour toutes
    public double total() { ... }
    public String rapport() { ... }
}
```

Deux banques, un seul traitement

Les deux banques livrent les mêmes informations, dans un ordre de colonnes différent

Seule la lecture d'une ligne change ; le total et le rapport sont communs

Une troisième banque coûtera une classe, et pas une ligne ailleurs

Banque	`ligne[0]`	`ligne[1]`	`ligne[2]`
Fric en Vrac	`compte`	`montant`	`communication`
Banquier Pépère	`communication`	`compte`	`montant`

Pourquoi `protected`

Une méthode que seule la classe mère appelle n'a pas à être `public`

Le modificateur `protected` l'ouvre aux sous-classes, et la ferme au reste

Une méthode abstraite `protected` reste redéfinissable par les sous-classes

La mesure de la réussite

La nouvelle classe devrait faire une vingtaine de lignes, dont trois méthodes de deux lignes

Si elle est plus longue, c'est que quelque chose a été recopié de la mère

Et le programme ne contient ni `instanceof`, ni condition sur la banque

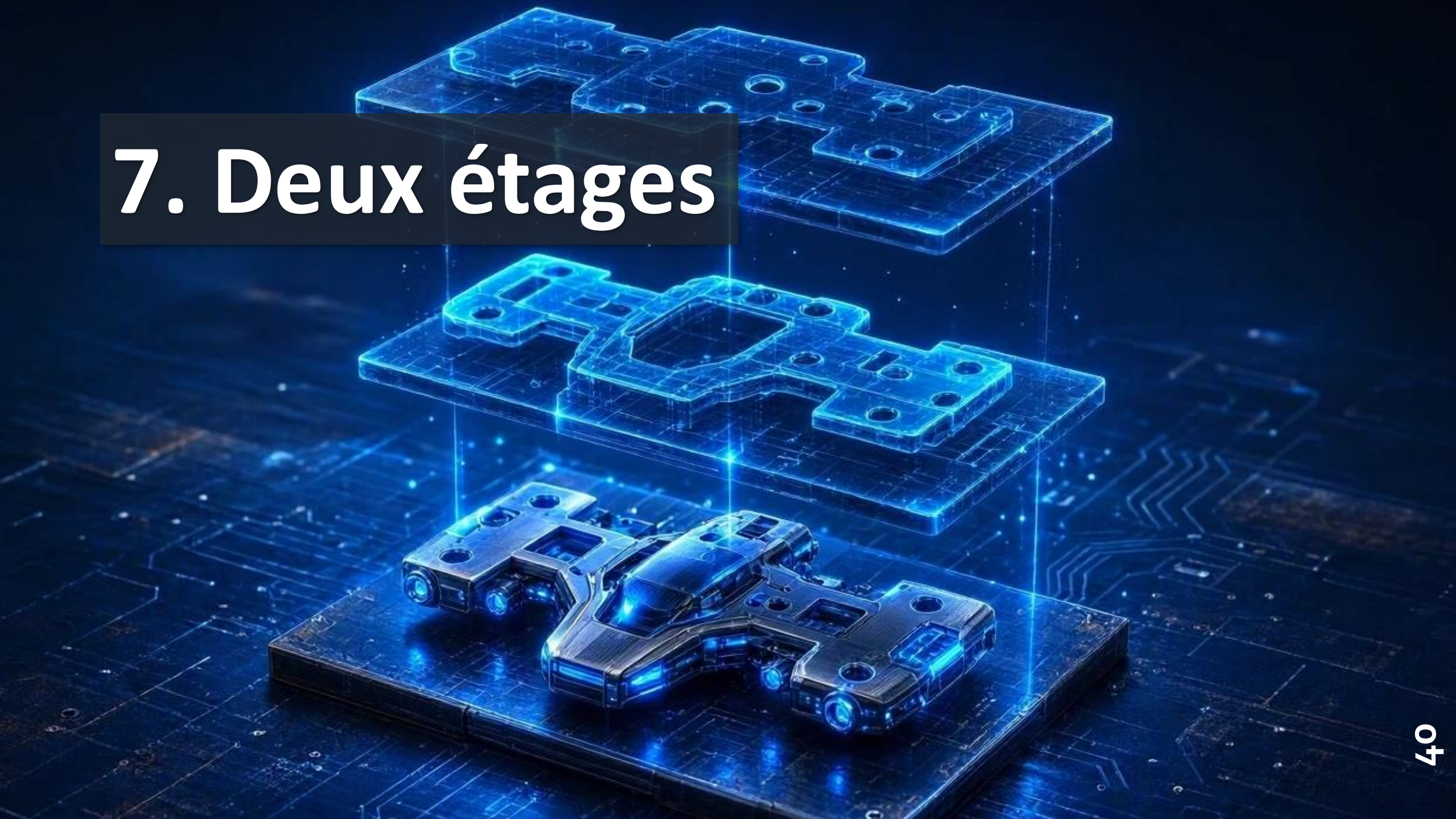
Exercices



ABS.3 Deux banques, un seul traitement

fascicule ABS

7. Deux étages



Une classe abstraite peut en étendre une autre

Rien n'interdit deux niveaux d'abstraction

Chaque étage contient ce qui est commun en dessous

Seules les feuilles de l'arbre sont concrètes

```
Facturable          interface : double montant()
  |
  +- Prestation     abstraite : client, facture()
      |
      +- Forfait    abstraite : prixForfait
      |   +- Abonnement
      |   +- Installation
      |
      +- Horaire    abstraite : tauxHoraire, heures
          +- Depannage
          +- Urgence
```

Où placer chaque champ

Un champ se déclare à l'étage où il devient commun

Trop haut, il encombre qui n'en a pas l'usage

Trop bas, il s'écrit deux fois

```
public abstract class Horaire extends Prestation {

    private final double tauxHoraire;
    private final int heures;

    protected double base() {
        return heures * tauxHoraire;
    }
}
```

Implémenter sans définir

Une classe abstraite peut implémenter une interface sans écrire ses méthodes

La promesse est reportée sur les sous-classes concrètes

La méthode `facture` appelle `montant`, qui n'existe encore nulle part

```
public abstract class Prestation implements Facturable {

    public String facture() {
        return client + " : " + montant() + " EUR";
    }
}
```

Exercices



ABS.4 Une hiérarchie à deux niveaux

fascicule ABS

À faire après le module ITF

8. Décider soi-même



Quand il n'y a plus de squelette

L'énoncé décrit ce que le programme doit faire, et s'arrête là

Combien de classes, laquelle est abstraite, où va chaque méthode : à vous de savoir

Les trois questions à se poser avant d'écrire une ligne

- Qu'est-ce que les classes ont en commun ?
- Ce qui est commun monte, et ne s'écrit qu'une fois
- Que sait faire la classe « en général » ?



Exercices

ABS.5 Tout écrire, à partir de rien

fascicule ABS

G.10 La classe abstraite `Jeu`

fil rouge GameHub



Attention à la borne : « jusqu'à 50 g » inclut les 50 g…